Desafío II: UdeABook
Análisis y diseño de una red social con programación orientada a objetos en C++

Jharlin Castro Moreno
Informática II, Semestre 2026-2
Facultad de Ingeniería, Universidad de Antioquia
Repositorio: https://github.com/jhacasky1524/Desafio2$_{U}deABook.git$

***Resumen*—Este documento presenta el análisis y el diseño de la capa lógica de UdeABook, una red social de consola desarrollada en C++ con programación orientada a objetos. Se describen el problema, las clases y sus relaciones, el diagrama de clases en UML simplificado, la selección de estructuras de datos propias con su análisis de eficiencia, el formato de los archivos de datos y los algoritmos esenciales. El diseño prioriza la eficiencia en iteraciones y memoria, empleando una matriz de adyacencia de bits para las amistades y listas dinámicas genéricas en lugar de contenedores de la STL.**

***Index Terms*—programación orientada a objetos, C++, diagrama de clases, estructuras de datos, matriz de adyacencia, eficiencia.**

I. Contextualización y análisis del problema

I-A. Objetivo y alcance

UdeABook es una red social cuyo núcleo son los usuarios, sus amistades, mensajes y álbumes de fotos. El desafío consiste en construir únicamente la capa lógica, en C++ con programación orientada a objetos (POO) y con interacción por consola. Quedan fuera de esta versión la interfaz gráfica, el registro de usuarios, la edición de álbumes y la eliminación de amistades. El criterio central de evaluación es la eficiencia, tanto en iteraciones como en memoria, por lo que el diseño parte de elegir estructuras de datos propias con memoria dinámica y de medir su costo.

I-B. Entidades del dominio

- **Usuario:** nombre de usuario único, fecha de nacimiento, correo, ruta de la foto de perfil, ciudad, país y fecha de inscripción.
- **Amistad:** relación mutua entre dos usuarios registrados.
- **Solicitud de amistad:** puede aceptarse o rechazarse; el receptor la conserva hasta resolverla.
- **Mensaje:** texto de hasta 512 caracteres con emisor, receptor, fecha y estado leído o no leído.
- **Álbum:** nombre, ruta de almacenamiento y privacidad (público, privado o solo amigos); contiene cero o más fotos.
- **Foto:** fecha de captura, tamaño, ruta del archivo `.png` y conteo de likes; pertenece a un único álbum.

I-C. Funcionalidades requeridas

El Cuadro I resume las funcionalidades y sus restricciones principales.

I-D. Restricciones técnicas

- Están prohibidos los contenedores de la STL, los `struct`, las librerías de terceros para lo esencial y las instrucciones de ANSI C. Se permite la librería de aleatorios y, según confirmó el profesor, `std::string`.
- Las estructuras de datos deben ser propias y usar memoria dinámica.
- La solución debe ser modular (archivos `.h` y `.cpp`) y contar con al menos tres clases.
- El almacenamiento permanente no se usa como área de procesamiento: los datos se cargan a memoria al iniciar y solo se escriben los cambios exigidos.
- Los datos estructurados se pasan por referencia para evitar copias innecesarias.
- Qt Creator se usa como entorno con un proyecto de consola, sin contenedores de Qt.

I-E. Supuestos y puntos por confirmar

El Cuadro II recoge los vacíos del enunciado y el supuesto adoptado en cada caso.

II. Identificación de clases y responsabilidades

Siguiendo la metodología del curso (sustantivos como clases y verbos como responsabilidades), se identificaron las clases del Cuadro III.

III. Diagrama de clases

Se aplica el UML simplificado del curso: una clase por caja, acceso con `+`, `−` y `#`, rombo del lado del todo en las relaciones «es parte de», nombre solo en la asociación simple y cardinalidad en todas las relaciones.

III-A. Versión resumen

III-B. Versión completa

III-C. Relaciones y cardinalidades

- **UdeABook ♦ Usuario (1 a N):** la plataforma contiene a todos los usuarios.

Cuadro I
FUNCIONALIDADES REQUERIDAS

Cód.	Funcionalidad	Restricciones relevantes
I	Carga y actualización de datos	No aparece en el menú; el almacenamiento permanente no se usa como área de procesamiento.
II	Ingreso y fotos de perfil de 3 amigos	Selección aleatoria ponderada, sin repetidos en el grupo y con opción de refrescar.
III	Administrar amistades	No enviar solicitud a amigos ni a quien ya envió una pendiente; aceptar solicitudes recibidas.
IV	Visitar perfil de un amigo	Solo álbumes públicos o de amigos; carrusel circular con salir, siguiente, previa y like.
V	Sugerencias de amistad	K entre 1 y 20, ordenadas por amigos en común, con opción de enviar solicitud.
VI	Administrar mensajes	Enviar (no vacío, máximo 512 caracteres), leer nuevos y buscar por rango de fechas inclusivo.
VII	Medición de recursos	Tras cada funcionalidad: iteraciones y memoria total en ese instante.

Cuadro II
SUPUESTOS Y PUNTOS POR CONFIRMAR CON EL CLIENTE

Punto	Supuesto adoptado
El enunciado habla de credenciales, pero el usuario no tiene contraseña entre sus atributos.	Pendiente de confirmar; si no hay contraseña, el inicio de sesión se hace solo con el nombre de usuario.
Dar o quitar like requiere saber quién ya lo dio.	Cada foto guarda la lista de índices de usuarios que dieron like; el conteo es el tamaño de esa lista.
Los rangos 10 o menos, 11 a 20 y más de 21 dejan sin cubrir el valor 21.	Se asume peso 1 para 10 o menos, 2,5 para 11 a 20 y 3,5 para 21 o más.
Amigos en común para la prioridad de fotos.	Se calculan entre el usuario autenticado y cada uno de sus amigos.
Fecha de los mensajes.	Se guarda fecha y hora para ordenar con precisión; el rango de búsqueda compara por día.

Cuadro III
CLASES Y RESPONSABILIDADES

Clase	Responsabilidad
`UdeABook`	Raíz del sistema: gestiona usuarios, sesión y coordina las funcionalidades.
`Usuario`	Guarda sus datos, álbumes, mensajes y solicitudes.
`Album`	Agrupa fotos y decide su visibilidad según la privacidad.
`Foto`	Guarda sus datos y la lista de likes.
`Mensaje`	Representa un mensaje y su estado de lectura.
`SolicitudAmistad`	Representa una solicitud pendiente.
`Fecha`	Valida y compara fechas.
`ListaDinamica<T>`	Plantilla de arreglo dinámico propio que reemplaza a la STL.
`MatrizAdyacencia`	Almacena la amistad con bits empaquetados.
`GestorArchivos`	Carga y guarda los datos permanentes.
`Medidor`	Cuenta iteraciones (miembro estático) y presenta las métricas.

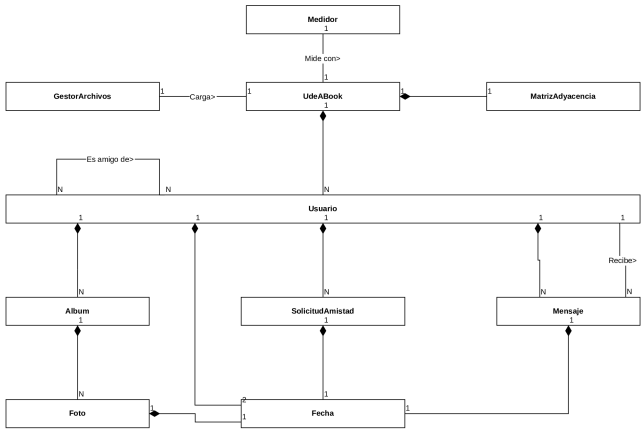


Figura 1. Diagrama de clases, versión resumen.

- **UdeABook ♦ MatrizAdyacencia (1 a 1):** una única matriz guarda las amistades de todos.
- **Usuario ♦ Album (1 a N)** y **Album ♦ Foto (1 a N):** un usuario puede no tener álbumes y un álbum puede no tener fotos; una foto pertenece a un único álbum.
- **Usuario ♦ Mensaje (1 a N):** el emisor es dueño de los mensajes que envía.
- **Usuario – Mensaje, «Recibe» (1 a N):** el receptor solo apunta al mensaje, de modo que existe una sola copia en memoria y el estado leído vive en un único lugar.
- **Usuario ♦ SolicitudAmistad (1 a N):** el receptor es dueño de las solicitudes que recibe.
- **Usuario – Usuario, «Es amigo de» (N a N):** asociación reflexiva respaldada por la matriz.
- **Fecha:** `Usuario` usa dos fechas (nacimiento e inscripción, cardinalidad 2); `Foto`, `Mensaje` y `SolicitudAmistad` usan una cada una.
- **UdeABook – GestorArchivos «Carga»** y **UdeABook – Medidor «Mide con»**: relaciones de uso, ambas 1 a 1.

`ListaDinamica<T>` aparece en la versión completa como tipo contenedor de los atributos; no se dibujan sus relaciones para evitar líneas redundantes.

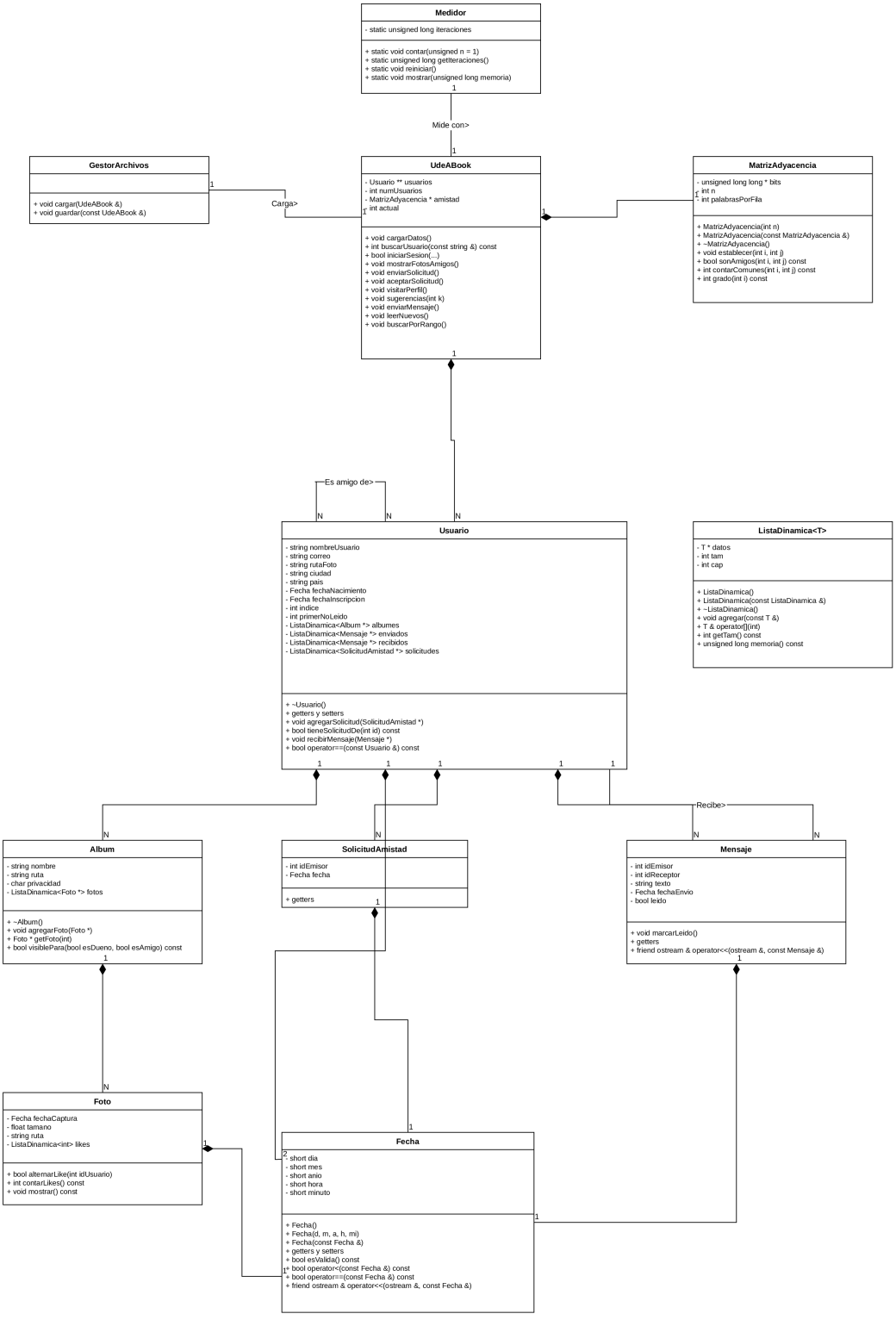


Figura 2. Diagrama de clases, versión completa.

III-D. Conceptos de POO aplicados

- **Encapsulamiento:** atributos privados con getters y setters como acceso controlado.
- **Constructor de copia y destructor:** `ListaDinamica`, `Fecha` y `MatrizAdyacencia`, con copia profunda en las clases que poseen memoria dinámica.
- **Sobrecarga de operadores:** `operator[]` en

`ListaDinamica`, `operator<` y `operator==` en `Fecha`, y `operator==` en `Usuario`.

- **Función amiga:** `operator«` para `Fecha` y `Mensaje`.
- **Plantilla:** `ListaDinamica<T>`.
- **Miembro estático:** el contador de iteraciones de `Medidor`.
- **Propiedad de la memoria:** quien crea un objeto con `new` es responsable de liberarlo con `delete`.

IV. Estructuras de datos y análisis de eficiencia

Sea n el número de usuarios (1000 o más), d los amigos de un usuario y m los mensajes. El Cuadro IV compara las alternativas consideradas.

V. Formato de archivos de datos

Se proponen archivos de texto con campos separados por punto y coma y un registro por línea. Las claves foráneas usan el nombre de usuario y el texto libre de los mensajes va siempre al final para tolerar caracteres especiales (Cuadro V).

Todos los archivos se cargan al inicio y solo se reescriben los datos que el enunciado permite actualizar: solicitudes, amistades, mensajes y likes. Si se confirma una contraseña, se agrega como campo de `usuarios.txt`.

VI. Algoritmos esenciales

Carga de datos. Se leen los usuarios y se ordenan por nombre; la posición final de cada uno es su índice en la matriz. Luego se leen las amistades marcando la matriz de forma simétrica, y después solicitudes, álbumes, fotos y mensajes, enlazando cada registro con su usuario mediante búsqueda binaria.

Inicio de sesión y fotos de perfil. Se busca el usuario por nombre. Para cada amigo se calcula la cantidad de amigos en común con la matriz y se asigna su peso (2, 5 o 7). Se elige un amigo con ruleta ponderada, se retira del sorteo y se repite hasta tener tres; si el usuario tiene menos de tres amigos, se muestran los que haya. Al refrescar se evita repetir los del grupo anterior cuando existan alternativas.

Enviar solicitud. Se valida que el destinatario exista, que no sea el mismo usuario, que no sean ya amigos, que no exista una solicitud previa enviada por el emisor y que este no tenga una pendiente del destinatario. Si todo es válido, se crea la solicitud en la lista del receptor.

Aceptar solicitud. Se marca la amistad en la matriz en ambos sentidos y se elimina la solicitud de la lista.

Visitar perfil y carrusel. Se comprueba que el nombre corresponda a un amigo y se listan los álbumes públicos o de amigos. Al elegir uno, la foto actual se recorre con un índice circular: siguiente suma 1 módulo el número de fotos y previa resta 1 con el mismo módulo. Dar o quitar like busca al usuario en la lista de likes de la foto y lo agrega o lo elimina.

Sugerencias de amistad. Con un arreglo de contadores de tamaño n, se recorren los amigos del usuario y, de cada uno, sus amigos, sumando una coincidencia a quienes no sean el usuario ni sus amigos. Luego se extrae el máximo K veces y se ofrece enviar solicitud a cualquiera de ellos.

Enviar mensaje. Se verifica que el destinatario sea amigo y que el texto no esté vacío ni supere 512 caracteres. El mensaje se crea una vez, queda en la lista de enviados del emisor y su puntero se agrega a la lista de recibidos del receptor.

Leer nuevos mensajes. Se recorre la lista de recibidos desde `primerNoLeido`, mostrando emisor y texto, y se marcan como leídos; el índice avanza al final.

Búsqueda por rango de fechas. Se hacen dos búsquedas binarias sobre los mensajes recibidos: la primera posición con fecha mayor o igual al inicio y la última con fecha menor o igual al fin. Se listan los mensajes entre ambas posiciones.

VII. Medición de recursos

Iteraciones. El miembro estático de `Medidor` se incrementa en cada pasada de cada ciclo, incluidos los de las estructuras internas como `ListaDinamica`, la matriz y las búsquedas. Se reinicia antes de cada funcionalidad y se muestra al terminar. Si se invoca un componente externo, se reporta su uso.

Memoria. Cada clase implementa un método que devuelve su tamaño real: el de sus atributos más la capacidad reservada de sus arreglos dinámicos, los objetos que posee y el contenido de sus cadenas. `UdeABook` suma el total de lo existente en el sistema y se agregan las variables locales y los parámetros por valor de la funcionalidad ejecutada.

VIII. Evolución del diseño y problemas de desarrollo

Evolución hasta la fecha. La estructura de los mensajes pasó de duplicarse en emisor y receptor a existir una sola vez, con el receptor apuntando a ella. La amistad se evaluó con listas y se cambió a una matriz de bits por el costo de calcular amigos en común. Para buscar usuarios se comparó una tabla hash y se eligió el arreglo ordenado por su simplicidad.

Problemas de desarrollo. [Se completará durante la implementación.]

Referencias

[1] Universidad de Antioquia, "Desafío II: Amigos para siempre, con UdeABook," Informática II, Semestre 2026-2, Oct. 2026.

[2] A. Guerra, "Programación orientada a objetos," diapositivas del curso Informática II, Universidad de Antioquia, 2026.

[3] Universidad de Antioquia, "Diagramas de clase," diapositivas del curso Informática II, 2026.

Cuadro IV
COMPARACIÓN DE ESTRUCTURAS Y COMPLEJIDAD

Operación	**Alternativas y costo**	**Elegida**
Buscar usuario por nombre	Recorrido lineal $O(n)$; arreglo ordenado con búsqueda binaria $O(\log n)$; tabla hash $O(1)$ promedio con manejo de colisiones.	Arreglo ordenado: sin registro de usuarios, se ordena una sola vez al cargar.
¿Son amigos?	Lista de amigos $O(d)$; matriz $O(1)$.	Matriz de adyacencia.
Amigos en común	Listas $O(d^2)$, u $O(d \log d)$ si están ordenadas; matriz de bits con AND de filas y conteo de bits, unas 16 operaciones de palabra para 1000 usuarios.	Matriz de bits (≈125 KB frente a ≈1 MB con `bool`).
Sugerencias (top K, $K \leq 20$)	Ordenar todos los candidatos $O(c \log c)$; contar y extraer el máximo K veces, $O(d^2) + O(Kn)$.	Contar y extraer K veces, pues K es pequeño.
Fotos de perfil ponderadas	Ruleta con pesos enteros 2, 5 y 7 (equivalentes a 1, 2,5 y 3,5), descartando al ya elegido: $O(d)$ por foto.	Ruleta sin reemplazo; los comunes se calculan una vez por amigo y se reutilizan al refrescar.
Mensajes por rango de fechas	Recorrido completo $O(m)$; dos búsquedas binarias $O(\log m + k)$.	Binaria, porque los mensajes se guardan en orden cronológico.
Leer mensajes no leídos	Recorrer todos $O(m)$; índice `primerNoLeido` $O(k)$.	Índice, ya que los no leídos quedan al final.
Crecimiento de `ListaDinamica`	Crecer de uno en uno, $O(n^2)$ copias en total; duplicar la capacidad, $O(1)$ amortizado.	Duplicar la capacidad.

Cuadro V
ARCHIVOS DE DATOS PROPUESTOS

Archivo	**Campos**
`usuarios.txt`	nombreUsuario; fechaNacimiento; correo; rutaFotoPerfil; ciudad; pais; fechaInscripcion
`amistades.txt`	usuarioA; usuarioB (un par por línea, sin duplicar el inverso)
`solicitudes.txt`	receptor; emisor; fecha
`mensajes.txt`	emisor; receptor; fechaHora; leido (0 o 1); texto
`albumes.txt`	dueño; nombreAlbum; rutaAlbum; privacidad (P, V o A)
`fotos.txt`	dueño; nombreAlbum; rutaArchivo; fechaCaptura; tamaño; listaDeLikes (usuarios separados por comas)